Dharmasala Animal Rescue Chatbot
High-Level Design (HLD) - Phase 1 with Phase 2 Extension

Document Version: 1.0 | Date: February 15, 2026
Prepared For: Project Director, Engineering Team, Rescue Operations
Prepared By: Kumar Abhinav (drafted for team review)

1) Executive Summary

Dharmasala Animal Rescue requires an AI-powered chatbot to assist the public and volunteers with animal rescue workflows, beginning with stray dog condition assessment from uploaded images and rescue guidance responses. The system should support triage, incident logging, duplicate detection, geolocation-aware escalation, safe conversational guardrails, and optional Slack integration for operational alerts.

This design proposes a scalable architecture for:
- Image-based triage using vision models
- Case record persistence in blob storage and relational database
- Duplicate and near-duplicate detection (exact plus perceptual similarity)
- Distress-triggered operational alerting (Slack/webhook)
- Guardrails and youth-friendly language
- Admin natural-language analytics (read-only, policy-controlled)

Phase plan:
- Phase 1 (MVP): English-only, image triage, incident history, alerts, admin analytics
- Phase 2: Hindi support, improved multilingual experiences, model tuning from real-world feedback

2) Business Context and Goals

Current Need:
Dharamshala Animal Rescue receives repeated rescue questions and image-based distress reports. Manual processing can delay urgent response and increase repeated-case noise.

Goals:
1. Provide immediate, safe first-response guidance for rescue-related queries.
2. Assess distress indicators from uploaded dog images.
3. Capture geolocation (when available) and escalate urgent cases.
4. Reduce duplicate case noise through automated similarity checks.
5. Enable operations teams to query incident trends using natural language.
6. Ensure safe behavior via strict AI guardrails and domain boundaries.

Non-Goals (Phase 1):
- Veterinary diagnosis or treatment prescription.
- Autonomous dispatch without human review.

- Full multilingual support (targeted in Phase 2).
- Human identity recognition from photos.

3) Stakeholders and User Personas

Primary Stakeholders:
- Project Director
- Engineering Lead
- Product Manager
- Rescue Operations Team
- Website Operations/Admin Team

User Personas:
1. Public Reporter (Youth/General Audience): uploads image and context, receives guidance.
2. Rescue Volunteer/Coordinator: reviews alerted cases and prioritizes field response.
3. Admin/Operations Analyst: uses dashboard and NL query interface for insights.

4) Key Use Cases

UC-1 Image-based Distress Assessment:
- User uploads stray dog image.
- System evaluates distress indicators and severity.
- Chatbot returns clear next steps and urgency level.

UC-2 Dog Bite Safety Guidance:
- User asks what to do after a bite.
- System provides immediate first-aid and escalation guidance (policy-aligned, non-diagnostic).

UC-3 Duplicate/Similar Case Detection:
- System identifies exact or likely duplicate incident.
- User response includes "similar report found" and status context.

UC-4 Distress Escalation to Operations:
- If severity threshold is met and location is available, send structured alert to Slack/webhook queue.

UC-5 Admin Natural-Language Analytics:
- Admin asks: "Show high-severity incidents in last 7 days by area."
- System converts NL to safe read-only SQL and returns summarized insights.

5) Functional Requirements

5.1 Chatbot and Interaction
- Accept text and image input via website widget and optional Slack integration.
- Provide youth-friendly, concise responses.
- Ask follow-up clarifying questions when confidence is low.

5.2 Vision Triage Pipeline
- Ingest image and metadata.
- Extract distress signals from image.

- Produce structured output: severity, confidence, indicators, recommended actions, escalation flag.

5.3 Geolocation Handling
- Attempt EXIF extraction when available.
- Fallback to browser geolocation consent or manual location input.
- Store location source and confidence.

5.4 Data Persistence
- Blob storage for image artifacts.
- SQL/MySQL for incidents, triage outputs, similarity links, and alert status.

5.5 Duplicate and Similarity Detection
- Exact duplicate check via SHA-256.
- Near-duplicate detection via perceptual hash and embedding similarity.
- Use confidence-based "potentially similar" messaging.

5.6 Alerting Workflow
- Distress threshold triggers alert workflow.
- Slack alert payload includes incident id, severity, confidence, location, and link.

5.7 Admin Analytics
- Read-only NL query layer.
- Safe query translation with strict allowlist and audit logs.

5.8 Guardrails
- Domain relevance filter.
- Harmful/inappropriate input detection.
- Medical-certainty limitation and legal-safe messaging.
- Prompt-injection protection.

6) Non-Functional Requirements

- Reliability: 99.5%+ availability target for MVP.
- Performance: P95 <4s for text query; P95 <8s for image triage (excluding upload variability).
- Scalability: horizontal API scaling and async queue-based heavy task handling.
- Security: TLS, encryption at rest, RBAC, secret vaulting.
- Privacy: data minimization, retention/deletion policy, auditability.
- Observability: traces, metrics, logs, alert dashboards.

7) Architecture Overview

Logical Components:
1. UI Layer: website chatbot widget; optional Slack interface.
2. API Layer: orchestration, auth/session, guardrail gateway, response builder.
3. AI Services: vision inference, safety classifier, response generator.
4. Incident Intelligence Layer: hashing/similarity, location resolver, severity policy engine.
5. Data Layer: blob storage, SQL/MySQL, optional vector index.

6. Ops Integration Layer: Slack/webhook notifier, admin NL-to-SQL broker.
7. Monitoring Layer: metrics, logs, traces, audit trails.

8) End-to-End Request Flows

Flow A: Image triage and response
1. User uploads image with optional text/location.
2. API validates file and payload.
3. Guardrails classify input and scope.
4. Blob storage write and SHA-256 generation.
5. EXIF parse and location fallback handling.
6. Vision inference for distress indicators.
7. Similarity checks (exact and near-duplicate).
8. Severity policy evaluation and escalation decision.
9. Incident persistence in SQL/MySQL.
10. User response with current condition, next steps, and similar-case context.
11. Slack alert generation if urgent.

Flow B: Text rescue question
- Domain classification, policy-safe response generation, emergency escalation steps when applicable.

Flow C: Admin NL query
- RBAC check, NL-to-SQL safe template resolution, read-only execution, audited response.

9) Data Model (High-Level)

Core Tables:

incidents:
- incident_id (UUID, PK)
- created_at, updated_at
- reporter_session_id (anonymized)
- image_blob_url
- image_sha256
- image_phash
- embedding_id (optional)
- lat, lng, location_source, location_accuracy
- triage_severity, triage_confidence, triage_summary
- distress_flags (JSON)
- similar_incident_id (nullable), similarity_score
- status (new/alerted/assigned/resolved/closed)

alerts:
- alert_id (UUID, PK)
- incident_id (FK)
- alert_channel (slack/webhook)
- trigger_reason

- sent_at
- ack_status, ack_by, ack_at

triage_events:
- event_id, incident_id, model_version
- raw_output_ref, postprocessed_output
- latency_ms, created_at

admin_query_audit:
- query_id, admin_user_id
- nl_query, resolved_sql_template
- executed_at, row_count, status

10) AI/ML Design Decisions

- Vision output is schema-constrained JSON.
- Post-processing enforces allowed labels and confidence bounds.
- Severity is policy-driven from cues, confidence, location, and repeat history.
- Similarity combines SHA-256 (exact), pHash (perceptual), and embedding nearest-neighbor score.
- Guardrails include intent classification, toxicity filters, medical/legal constraints, and injection defenses.

11) API Surface (Illustrative)

Public APIs:
- POST /v1/triage/image
- POST /v1/chat/query
- POST /v1/location/update
- GET /v1/incidents/{incident_id}

Admin APIs:
- POST /v1/admin/query
- GET /v1/admin/incidents
- GET /v1/admin/alerts
- POST /v1/admin/incidents/{incident_id}/status

Integration APIs:
- POST /v1/integrations/slack/alert
- POST /v1/integrations/events

12) Security, Privacy, and Safety Controls

Security:
- JWT/session validation and RBAC.
- Secret management via vault/KMS.
- Signed URLs for temporary image access.

Privacy:
- No unnecessary personal identity storage.
- Location precision truncation where needed.
- Data retention and purging policy.

Safety:
- Non-diagnostic language and escalation-first guidance.
- Emergency playbooks for high-risk scenarios.
- Misuse detection and safe refusal behavior.

13) Observability and SRE Plan

Metrics:
- Traffic, latency, errors, model latency/failures.
- Severity distribution, duplicate rates, alert acknowledgment time.
- Guardrail trigger frequency.

Logs:
- Structured request and inference logs.
- Admin query and security audit logs.

Operational Alerts:
- API/model failure spikes.
- Slack dispatch failures.
- Database health degradation.

14) Deployment and Environments

Environments:
- dev for rapid iteration
- staging for UAT and pre-prod validation
- prod for controlled rollout

Deployment:
- Containerized services.
- CI/CD gates: unit tests, API contract tests, guardrail regression tests.
- Canary strategy for model or prompt changes.

15) Testing Strategy

- Functional tests: upload, triage, persistence, and alert workflows.
- Guardrail tests: off-topic, abusive input, prompt injection, policy edge cases.
- Model evaluation: severity agreement with human reviewers, false-negative tracking.
- Performance tests: concurrency, P95/P99 latency.
- UAT: volunteer review for actionability and youth readability.

16) Rollout Plan

Phase 1A (Weeks 1-3):
- Core APIs, image ingestion, storage, basic triage, incident persistence

Phase 1B (Weeks 4-6):
- Similarity detection, Slack alerts, guardrails, admin query beta

Phase 1C (Weeks 7-8):
- Hardening, observability, UAT, limited production pilot

Phase 2:
- Hindi support
- Multilingual guardrails
- Active-learning feedback loops
- Expanded rescue knowledge coverage

17) Risks and Mitigations

1. Inaccurate vision interpretation
- Mitigation: conservative thresholds and human-review escalation.

2. Missing location data
- Mitigation: browser/manual location fallback prompts.

3. False duplicate linking
- Mitigation: confidence-based messaging and human-review linkage.

4. Guardrail bypass attempts
- Mitigation: layered moderation and policy enforcement.

5. Alert fatigue
- Mitigation: severity gating and de-dup alert windows.

6. Admin query misuse
- Mitigation: read-only templates, allowlists, limits, and audit logs.

18) Governance and Ownership

- Product Owner: scope, roadmap, prioritization.
- Engineering Lead: architecture and delivery.
- ML Lead: model quality and safety threshold governance.
- Ops Lead: response playbooks and SLA adherence.
- Security/Compliance: policy and audit readiness.

19) Success Metrics

User Outcomes:
- >=80% users receive actionable first response under 10s.
- >=70% pilot users rate response clarity/usefulness positively.

Operational Outcomes:
- >=40% reduction in mean alert acknowledgment time.
- >=25% reduction in duplicate-case operational noise.

Quality and Safety:
- High-severity false negatives remain below approved threshold.
- >95% guardrail containment for out-of-scope/harmful prompts.

20) Open Decisions for Director Review

1. Final cloud platform and storage choices.
2. Slack integration scope in MVP vs post-MVP.
3. Data retention window (90/180/365 days).
4. Auto-alert severity threshold and policy.
5. Admin NL analytics rollout timing (MVP vs controlled beta).
6. Phase 2 Hindi scope and staffing.

Appendix A: Example Youth-Friendly Response

Potential Distress Detected (High Priority):
"I can see signs that this dog may need urgent help. Please keep a safe distance, avoid sudden movements, and contact rescue support now. If possible, share the exact location or nearest landmark so volunteers can reach quickly."

Appendix B: Example Slack Alert Payload

- incident_id
- timestamp
- severity and confidence
- distress_indicators
- location/map_link
- similar_incident_reference
- admin_console_url